Washington State University Vancouver

Information Retrieval - CS 454

Assignment 1

Professor:
Ben McCamish

Overall Assignment

The goal of this assignment is to create a web crawler using Python 3.6+. To ensure that all students are crawling available data and not spamming the servers, always practice safe crawling techniques. Once you have collected the data, it must be stored in a directory. For turn in, do not turn in the entirety of what you crawled, but instead a small subset. Feel free to come show me some data partway through the assignment to make sure that the amount collected is acceptable. As a general target, you should try for at least 5000 pages. It is much better if you can get more. Make your crawler generic so if you need to (hint: you will) crawl some 100,000 pages later, it will be easy.

Seeding your crawler

First, decide on a URL that you will use to seed your crawler. It needs to start somewhere. I suggest a page that has a lot of outgoing links. WSU's homepage for instance has many outgoing pages. You may also use Wikipedia as a starting point to find some of their outgoing links.

Crawling

Once you have decided on a place to start, you can begin to crawl. You may use the algorithm covered in class, but feel free to explore and find alternative methods. Be sure to get all additional libraries approved by the professor.

When crawling, you need to collect multiple things. First, the raw text of the page. This means you don't want the entire HTML file, but rather, just the important bits of text. Second, the URL of the page you got the URL from. You will need the URL to make sure you don't crawl the same page twice. You will also need the URL to ensure you can retrieve the original page again. Finally, when crawling, keep an adjacency matrix so you know which pages link to others. This will be directed, as a page might point to another, but not visa versa.

Output

When I run your code, there should be clean output on the terminal showing what is currently going on. Your code must also store the required elements to disk in whatever way you chose. Be sure to outline how this happens in the writeup.

Some Notes: Be sure to respect the robots.txt on each domain you crawl. You may want to start by only crawling pages within your seed domains. If that works, then you can generalize your crawler and find pages on other domains. You may want to start this project early so you have time to let it run and crawl a decent number of pages once you have worked out the bugs.

Remember, while it may not take many lines of code to complete this project, it will take some time. We are dealing with real-world data which is often imperfect and requires fine tuning. Run your code in a variety of domains to ensure you have captured the edge cases.

To Receive full marks you must:

- (5%) Specify the domains you are crawling.
- (40%) Write a python program that crawls the web for at least 5000 pages. Your code should be generalized so the number of pages crawled can be adjusted. You should also provide a nice interface, either command line or via method calls, that is described in the README.
- (30%) Keep track of what URLs you have visited and an adjacency matrix of the connection between pages. I suggest you use some external files to store these once your program has finished, so you can pause and resume your code. Something like XML, JSON, CSV, etc. You might also look at Pickle in python, so you can save and load objects.
- (25%) A writeup, written in LaTeX, explaining your project. This includes a list of seeds, a sample of the pages your scraped, how you decided to store the matrix and list of URLs, and how your program can be expanded. In your writeup discuss the process of development, how long does it take to run, what are some bugs you encountered, what did you find the most difficult about the project, what did you find the most interested, and anything else you think might be useful to mention.

What to turn in (in a zip on Canvas):

- All code (well commented)
- A sample of your crawled pages (5 or so)
- A summary of your project, both the LaTeX source and a PDF.
- README.txt on how to run your code (detailed if required).